

Caching

In This Edition

1	Overview --- What It Is	3
2	Why It Exists	3
	2.1 Locality --- why caches work at all	4
	2.2 Hit-ratio math --- the core formula	4
	2.3 Load reduction --- the other half	4
3	Why FAANG Cares	5
4	Core Concepts	5
5	Caching Patterns	6
	5.1 Cache-aside (lazy loading) --- the default	6
	5.2 Read-through	7
	5.3 Write-through	7
	5.4 Write-back (write-behind)	8
	5.5 Write-around	8
	5.6 Comparison table	8
6	Eviction Policies	9
	6.1 LRU --- full O(1) implementation	9
	6.2 LFU --- counts, and the aging problem	10
	6.3 FIFO / MRU	10
	6.4 ARC (Adaptive Replacement Cache)	11
	6.5 W-TinyLFU --- what Caffeine uses	11
	6.6 Scan resistance --- why it matters	11
	6.7 Redis maxmemory-policy options	11
7	Cache Invalidation	12
	7.1 1. TTL expiry --- the workhorse	12
	7.2 2. Write-time invalidation: DELETE, not UPDATE	12
	7.3 3. CDC / binlog-driven invalidation	12
	7.4 4. Versioned keys	13
	7.5 5. Stale-while-revalidate (SWR)	13
	7.6 The read-after-write inconsistency window	13
8	The Thundering Herd & Friends (Failure Modes)	14
	8.1 Cache stampede (thundering herd / dog-pile)	14
	8.2 Cache penetration	15
	8.3 Cache avalanche	16
	8.4 Hot key	16
9	Redis Deep Dive	17
	9.1 Data structures (and what to cache with each)	17
	9.2 Single-threaded model + pipelining	18
	9.3 Persistence: RDB vs AOF	18
	9.4 Replication & Sentinel	19
	9.5 Redis Cluster --- 16384 hash slots	19
	9.6 Distributed locks: Redlock and its criticism	19
	9.7 Rate limiting in Redis	19
	9.8 Pub/Sub	20
	9.9 Redis vs Memcached	20
10	CDN & HTTP Caching	20
	10.1 CDN (Content Delivery Network)	20
	10.2 HTTP caching headers --- the protocol	21
	10.3 ETag + conditional requests → 304	21
	10.4 Vary and Age	22
	10.5 Browser vs CDN vs origin	22
11	Consistency & Multi-Region	22
	11.1 Staleness windows --- name them, bound them	22
	11.2 Cache coherence across nodes/regions	22
	11.3 When NOT to cache	23

12	Designing a Cache (Worked Example)	23
12.1	1. Clarify access patterns & SLOs	23
12.2	2. Choose tiers	23
12.3	3. Keys, values, TTLs	23
12.4	4. Invalidation	24
12.5	5. Stampede / avalanche / hot-key protection	24
12.6	6. Expected hit ratio & load math	24
13	Architecture / Diagrams	24
13.1	Cache-aside read/write flow	24
13.2	Multi-tier cache hierarchy	25
13.3	Stampede: herd vs single-flight	25
13.4	Where caches live (request path)	25
14	Real-World Examples	25
15	Real-Life Analogies	26
16	Memory Tricks / Mnemonics	27
17	Common Interview Questions	27
17.1	Q1: Explain cache-aside, and when would you choose it over write-through?	27
17.2	Q2: How do you keep a cache consistent with the database?	27
17.3	Q3: What is a cache stampede (thundering herd) and how do you prevent it?	28
17.4	Q4: How do you protect against queries for keys that don't exist (penetration)?	28
17.5	Q5: LRU vs LFU --- implement LRU and explain when each wins.	28
17.6	Q6: Where would you place caches in a web architecture, and what does each buy you?	28
17.7	Q7: Walk me through Redis internals you'd rely on for a cache.	29
17.8	Q8: How do distributed locks work in Redis, and what are the caveats?	29
17.9	Q9: Explain HTTP caching: Cache-Control, ETag, and what 304 saves.	29
17.10	Q10: Design caching for a product catalog under a flash sale.	29
17.11	Q11: Your cache just went down in production. What happens, and how should the system behave?	30
17.12	Q12: How do you choose a TTL?	30
18	Senior-Level Discussion Points	30
19	Typical Mistakes Candidates Make	31
20	How This Connects to Other Topics	31
21	FAANG Interview Tips	32
22	Revision Cheat Sheet	32
22.1	10-Minute Summary	32
22.2	Key Numbers to Know	33
22.3	Cheat-Sheet Table	33

How to use this file: Read top-to-bottom for deep mastery. "Add a cache" is the most common scaling answer in system design — but the senior signal is the *details*: which pattern, which eviction policy, how you invalidate, and how you survive a stampede. Jump to §Common Interview Questions + §Revision Cheat Sheet for last-minute revision. The hard part is never the cache hit — it's **invalidation** and **failure modes**.

1 Overview --- What It Is

A **cache** is a fast, usually in-memory store of frequently-accessed or expensive-to-compute data, placed in front of a slower source of truth (database, microservice, disk, or a remote API) to cut **latency** and **load**.

The fundamental trade is **freshness for speed**: a cache returns a *possibly-stale* copy quickly instead of the authoritative-but-slow original. Caching works because of **locality** — in almost every real workload a small fraction of the data receives most of the requests (a Pareto / power-law / Zipfian distribution). Cache that small hot set and you absorb the majority of traffic in microseconds.

```
Without cache: Client —————> DB          every read hits the DB (~5-50 ms)
With cache:   Client -> Cache (hit, ~0.2 ms) -> return
               |
               | miss -> DB -> fill cache -> return
```

Term	Meaning
Hit	Requested key found in cache — served fast
Miss	Not found — fall through to the source, then populate ("fill")
Hit ratio	hits / (hits + misses) — the single most important cache metric
TTL	Time-to-live; after it expires, the entry is treated as absent
Eviction	Removing an entry to make room when the cache is full
Invalidation	Removing/refreshing an entry because the source changed
Working set	The data actively used in a time window; the cache should fit it

A cache is **not a database**. It is allowed to forget. The defining property of a cache is that *losing the cache must never lose data and must never break correctness* — only performance. The moment you depend on the cache for durability, you have built a fragile database, not a cache.

2 Why It Exists

Memory is fast; everything else is slow. The whole justification for caching lives in the **latency ladder** — the orders-of-magnitude gaps between layers of the memory/storage/network hierarchy.

Operation	Latency	Relative to L1	Human-scale (×1B)
L1 cache reference	~0.5 ns	1×	0.5 sec
Branch mispredict	~5 ns	10×	5 sec
L2 cache reference	~5 ns	10×	5 sec
L3 / LLC reference	~20 ns	40×	20 sec
Main memory (DRAM) reference	~100 ns	200×	100 sec (~1.5 min)
Read 1 MB sequentially from RAM	~3 μs		~50 min
Redis GET over LAN (in DC)	~0.1-0.5 ms	200,000-1M×	1-5 days

SSD random read (NVMe)	~100 μ s	200,000×	~1 day
Round trip within same DC	~0.5 ms	1M×	~5.8 days
Read 1 MB sequentially from SSD	~1 ms		~11 days
Disk (HDD) seek	~5–10 ms	10M–20M×	~6 months
Round trip CA → Netherlands	~150 ms	300M×	~9.5 years

The takeaway: a **DRAM/Redis read is 100–1000×** faster than a database query that touches disk, and the database query may itself involve parsing, planning, locking, B-tree traversal, and network hops. If the same answer is requested many times, recomputing or re-reading it is pure waste.

2.1 Locality --- why caches work at all

- ▶ **Temporal locality** — data used recently is likely to be used again soon. (You refresh your own profile page 10 times in a session.) → keep recently-used keys.
- ▶ **Spatial locality** — data near recently-used data is likely to be used. (Reading row 100 → you'll probably read rows 101–110.) → fetch and cache in blocks/pages.
- ▶ **Popularity skew (Zipf)** — the k -th most popular item gets traffic $\propto 1/k$. The top 20% of keys often serve 80%+ of requests. This is *why* a cache one-tenth the size of the dataset can still hit 90% of the time.

2.2 Hit-ratio math --- the core formula

The **effective (average) latency** of a read path with a cache:

```

1 T_effective = h * t_cache + (1 - h) * t_miss
2
3 where h      = hit ratio (0..1)
4     t_cache  = latency of a cache hit
5     t_miss   = latency of a miss (cache lookup + DB fetch + fill)

```

Worked numbers. Say $t_{\text{cache}} = 0.5$ ms, $t_{\text{db}} = 20$ ms, so $t_{\text{miss}} \approx 0.5 + 20 = 20.5$ ms.

Hit ratio h	$T_{\text{effective}}$	Speedup vs no cache (20 ms)
0% (no cache)	20.0 ms	1.0×
50%	10.5 ms	1.9×
80%	4.5 ms	4.4×
90%	2.5 ms	8.0×
95%	1.5 ms	13×
99%	0.7 ms	29×
99.9%	0.52 ms	38×

Notice the curve: the jump from 90% → 99% ($8\times \rightarrow 29\times$) is enormous. **The last few percent of hit ratio matter most** — this is why teams obsess over shaving misses. It is an Amdahl's-Law-style argument: the misses are the serial fraction that dominates as $h \rightarrow 1$.

2.3 Load reduction --- the other half

Hit ratio also dictates **how much load reaches the DB**. If reads arrive at 100,000 QPS and $h = 0.95$, the DB sees only $(1 - 0.95) \times 100,000 = 5,000$ QPS. A 20 $\times$ reduction. **This is usually the real reason to cache** — not user latency, but protecting a database that physically cannot serve 100K QPS from melting down. Raising h from 95% → 99% drops DB load from 5,000 → 1,000

QPS, a 5× further reduction on the part that actually hurts.

Caching is the highest-leverage scaling lever for read-heavy systems — and the hardest to keep correct. A 95% hit ratio means a 1-node DB can pretend to be a 20-node DB.

3 Why FAANG Cares

- › **System design ubiquity** — “the DB is the bottleneck” → “add a cache” is a near-universal move. The interviewer immediately probes the *details*: pattern, TTL, invalidation, stampede protection, hit-ratio estimate. A vague “I’d use Redis” scores nothing; a precise “cache-aside on product:{id} with a 5-min TTL plus 10% jitter, delete-on-write, single-flight on miss, expecting ~92% hit ratio” signals seniority.
- › **The famous hard problem** — “*There are only two hard things in computer science: cache invalidation and naming things.*” (Phil Karlton). Reasoning crisply about staleness windows is a senior signal.
- › **Real production impact at FAANG:**

Company	Caching reality
Meta	memcached fronts MySQL at massive scale; the “ <i>Scaling Memcache at Facebook</i> ” paper introduces leases (anti-stampede + anti-stale-set), regional pools, and the “look-aside” architecture. TAO is a graph cache over MySQL.
Netflix	EVCache (a Memcached-based, multi-region replicated cache) absorbs huge read volume; sized so a full region failover still has warm caches.
Amazon	ElastiCache (Redis/Memcached) + DAX (in-front-of-DynamoDB cache) + CloudFront CDN. DynamoDB itself is partly a giant managed KV cache pattern.
Google	CPU caches → memcache-style tiers → edge caching for Search/YouTube. The “Tail at Scale” paper shows why a single cold cache miss in a fan-out request poisons p99.
Cloudflare/Akamai/Fastly	Entire business is caching (CDN at the edge).

4 Core Concepts

- › **Hit / miss / fill** — found vs not; a miss triggers a fetch from the source and a *fill* (populate) of the cache.
- › **TTL (time-to-live)** — expiry after which an entry is stale and must be refetched. The cheapest invalidation mechanism.
- › **Hot vs cold keys** — frequently vs rarely accessed. Caches keep the hot set; cold keys churn.
- › **Working set** — the data touched in a window. If the working set fits in cache memory, hit ratio is high; if not, you thrash.
- › **Cache warming / cold start** — an empty cache (post-deploy, post-restart) has 0% hit ratio; every request misses and the DB takes the full load. Mitigate with pre-warming.
- › **Consistency vs freshness** — how stale a read may be. Tuned *per use case*: a stale stock price is dangerous, a stale avatar is harmless.
- › **Negative caching** — caching the fact that a key does *not* exist (to stop penetration).
- › **Read/write amplification** — extra ops caused by fills and invalidations (e.g., a celebrity post invalidating 100M follower feed entries).
- › **Cardinality** — number of distinct keys. High cardinality + low reuse = poor cache fit.

5 Caching Patterns

The pattern defines **who reads/writes the cache and the DB, and in what order**. This is the single most-asked caching topic. Get the read flow, the write flow, and the race conditions for each.

CACHE-ASIDE (look-aside / lazy)	READ-THROUGH
Read: app→cache(miss)→db→fill→return	Read: app→cache lib→(miss)→db→fill (lib does it)
Write: app→db, then DELETE cache key	Write: (separate; often write-through)
WRITE-THROUGH	WRITE-BACK (write-behind)
Write: app→cache→db (both, sync)	Write: app→cache (return), async flush→db
Read: app→cache (always fresh)	Risk: data loss on crash before flush
WRITE-AROUND	
Write: app→db (skip cache)	
Read: app→cache(miss)→db→fill	

5.1 Cache-aside (lazy loading) --- the default

The application is in charge. The cache is a "dumb" KV store that knows nothing about the DB.

```

1 def get_user(user_id):
2     key = f"user:{user_id}"
3     val = cache.get(key)
4     if val is not None:           # HIT
5         return deserialize(val)
6     # MISS
7     row = db.query("SELECT * FROM users WHERE id = %s", user_id)
8     if row is not None:
9         cache.set(key, serialize(row), ex=300) # fill with 5-min TTL
10    return row
11
12 def update_user(user_id, changes):
13     db.update("users", user_id, changes) # 1. write the source of truth
14     cache.delete(f"user:{user_id}")     # 2. invalidate (DELETE, not update)

```

Why it's the default:

- › **Resilient** — if Redis is down, reads fall through to the DB. Cache outage = slower, not broken.
- › **Lazy** — only data that's actually requested ends up cached; you don't waste memory on cold rows.
- › **Simple mental model** — the app owns both paths.

Costs:

- › **First read is always a miss** (cold key) → one extra round trip + fill.
- › **Staleness window** between a DB write and the cache delete.
- › **The stale-read race** (below) — the classic cache-aside correctness bug.

The cache-aside stale-read race (draw this in the interview)

Two operations interleave: a reader filling the cache after a miss, and a writer updating the DB and deleting the key.

Time	Reader R (reading user:42)	Writer W (updating user:42)
t1	GET user:42 → MISS	
t2	SELECT users WHERE id=42 → v1	

```

                                UPDATE users SET ... → v2 (commit)
t3                                DELETE user:42      (cache empty)
t4      SET user:42 = v1  ▲  STALE!
-----
Result: cache now holds v1 forever (until TTL), DB holds v2.

```

R read the *old* value at t2, W committed the *new* value and deleted the (already-empty) key at t3, then R *re-populated the cache with the stale v1* at t4. The cache is now wrong until the TTL expires.

Mitigations (in order of practicality):

1. **TTL as a backstop** — the stale value self-heals after the TTL. Cheap, bounds the damage. Usually "good enough."
2. **Facebook-style leases** — on a miss, the cache hands the reader a *lease token*. A write invalidates outstanding leases, so a stale fill is rejected. (From the Meta memcache paper.) This closes the race.
3. **Delete-after-delay (double delete)** — writer deletes the key, and schedules a *second* delete ~1 second later to catch a stale fill that landed in between. Pragmatic and common.
4. **Versioning / CAS** — store a version with the value; only fill if the version is still current.

Interview takeaway: Cache-aside has a real read-after-write race. Knowing it exists — and that **TTL bounds it** while **leases/double-delete close it** — is a strong senior signal. Most candidates don't know this race exists.

5.2 Read-through

Like cache-aside, but the *cache library/proxy* owns the miss→DB→fill logic, not the app. The app just calls `cache.get(key)` and a configured loader function fetches from the DB transparently (e.g., Caffeine `LoadingCache`, DynamoDB DAX). Cleaner app code, but couples the cache to the data source and you lose some control over the miss path.

```

1 LoadingCache<Long, User> cache = Caffeine.newBuilder()
2   .maximumSize(100_000)
3   .expireAfterWrite(Duration.ofMinutes(5))
4   .refreshAfterWrite(Duration.ofMinutes(1)) // stale-while-revalidate-ish
5   .build(userId → db.loadUser(userId));    // the loader = read-through
6
7 User u = cache.get(42L); // returns cached, or loads via the loader on miss

```

5.3 Write-through

Every write goes to **cache and DB synchronously**. The cache is always fresh for what's been written.

```

1 def update_user(user_id, changes):
2     row = db.update("users", user_id, changes) # write DB
3     cache.set(f"user:{user_id}", serialize(row), ex=300) # write cache (same txn-ish)
4     return row

```

- **Pro:** reads after a write are always fresh; no stale-read race on written keys.
- **Con:** higher write latency (two writes on the critical path); you cache data that may *never be read* (wasted memory); doesn't help cold reads (key only cached if written).

- › **Note:** if the cache write fails after the DB write, you have an inconsistency — usually solved by treating cache as best-effort and relying on TTL, or by deleting instead of setting.

5.4 Write-back (write-behind)

Write to the **cache only**, return immediately, and flush to the DB **asynchronously** (batched).

```
app → cache (ACK now) └─ background flusher ─(batch every 100ms / 1000 writes)→ DB
```

- › **Pro:** lowest write latency, highest write throughput, batches/coalesces writes (10 updates to the same key → 1 DB write). Great for high-frequency counters, view counts, "last seen" timestamps.
- › **Con / the durability risk:** if the cache node crashes before the flush, **those writes are lost**. The cache temporarily *is* the source of truth — exactly the trap. Use only where loss of a few seconds of writes is acceptable (analytics counters), or pair with a durable write-ahead log (Redis AOF, Kafka).
- › Used inside OS page caches and SSD controllers, and by metric pipelines.

5.5 Write-around

Writes go **straight to the DB, bypassing the cache**. The cache is populated only on read (lazy).

- › **Pro:** avoids polluting the cache with write-heavy data that won't be read soon (e.g., bulk imports, logs). Keeps the cache focused on the read-hot set.
- › **Con:** a read immediately after a write is a guaranteed miss (the value isn't cached yet).

5.6 Comparison table

Pattern	Write path	Read path	Consistency	Write latency	Best for
Cache-aside	DB, then delete key	cache → miss → DB → fill	Eventual (TTL-bounded)	Low	Default; read-heavy, tolerable staleness
Read-through	(paired separately)	cache lib loads on miss	Eventual	Low	Clean code; cache owns the load
Write-through	cache + DB sync	cache (always fresh)	Strong on written keys	Higher (2 writes)	Read-heavy, low staleness tolerance
Write-back	cache, async → DB	cache	Eventual + loss risk	Lowest	Write-heavy counters, can tolerate loss
Write-around	DB only	cache → miss → DB → fill	Eventual	Low	Write-heavy data rarely re-read

Which pattern for which workload?

- › **Read-heavy, staleness OK** (product catalog, profiles, feeds) → **cache-aside** (with TTL).
- › **Read-heavy, must be fresh** (config, feature flags) → **write-through** or cache-aside + CDC invalidation.
- › **Write-heavy, ephemeral** (counters, sessions, rate limits) → **write-back** or just use Redis as the store with AOF.
- › **Write-heavy, rarely re-read** (event logs, bulk loads) → **write-around**.

Interview takeaway: Default to **cache-aside**. Reach for write-through only when you can't tolerate the stale-read window, and write-back only when you can tolerate data loss. Always name the trade-off out loud.

6 Eviction Policies

When the cache is full and a new key must be inserted, **which existing entry do we drop?** Eviction policy is what makes a fixed-size cache approximate an infinite one. The goal: evict the entry *least likely to be used again* (Bélády's optimal algorithm evicts the one used furthest in the future — unknowable, so we approximate).

Policy	Evicts	Pros	Cons
LRU	least <i>recently</i> used	great for temporal locality; $O(1)$	one big scan flushes the hot set (scan pollution)
LFU	least <i>frequently</i> used	keeps genuinely popular items; scan-resistant	needs counters; can keep stale-popular items without aging
FIFO	oldest <i>inserted</i>	trivial; $O(1)$	ignores access pattern; can evict hot items
MRU	most <i>recently</i> used	good when recent = won't reuse (e.g. sequential scans)	rare; counter-intuitive
Random	a random entry	$O(1)$, no metadata; surprisingly OK at scale	no locality awareness
TTL / volatile	expired (or soonest-to-expire)	freshness-bound data	not about capacity per se
ARC	adaptive recency+frequency	self-tunes; scan-resistant	patented (historically); more memory
W-TinyLFU	frequency-gated admission + LRU	near-optimal hit ratios; scan-resistant	more complex (Caffeine)

6.1 LRU --- full $O(1)$ implementation

LRU is the canonical interview implementation: a **hash map** for $O(1)$ lookup + a **doubly-linked list** for $O(1)$ recency ordering. Most-recently-used at the head, least at the tail. On access, move the node to the head; to evict, drop the tail.

```

Python
1 class Node:
2     __slots__ = ("key", "val", "prev", "next")
3     def __init__(self, key, val):
4         self.key, self.val = key, val
5         self.prev = self.next = None
6
7 class LRUCache:
8     def __init__(self, capacity):
9         self.cap = capacity
10        self.map = {} # key → Node (O(1) lookup)
11        # sentinel head/tail to avoid null checks
12        self.head = Node(None, None) # MRU side
13        self.tail = Node(None, None) # LRU side
14        self.head.next = self.tail

```

```

15     self.tail.prev = self.head
16
17     def _remove(self, node):
18         node.prev.next = node.next
19         node.next.prev = node.prev
20
21     def _add_front(self, node):      # insert right after head (MRU)
22         node.next = self.head.next
23         node.prev = self.head
24         self.head.next.prev = node
25         self.head.next = node
26
27     def get(self, key):
28         node = self.map.get(key)
29         if node is None:
30             return -1                # MISS
31         self._remove(node)          # touch → move to front
32         self._add_front(node)
33         return node.val
34
35     def put(self, key, val):
36         if key in self.map:
37             node = self.map[key]
38             node.val = val
39             self._remove(node)
40             self._add_front(node)
41             return
42         if len(self.map) ≥ self.cap:    # evict LRU (tail.prev)
43             lru = self.tail.prev
44             self._remove(lru)
45             del self.map[lru.key]
46         node = Node(key, val)
47         self.map[key] = node
48         self._add_front(node)

```

Doubly-linked list (recency order):

```

head → [k9] → [k4] → [k7] → ... → [k2] → tail
      MRU                      LRU
get(k7) → unlink k7, splice after head:
head → [k7] → [k9] → [k4] → ... → [k2] → tail
put when full → drop tail.prev (k2), the least-recently-used.

```

Every operation is $O(1)$. (In Java: `LinkedHashMap` with `accessOrder=true` and an overridden `removeEldestEntry` gives you LRU for free.)

6.2 LFU --- counts, and the aging problem

LFU evicts the entry with the **lowest access count**. A clean $O(1)$ LFU uses a hash map `key→(value, freq)` plus a map `freq→doubly-linked-list-of-keys` and a `min_freq` pointer.

The flaw: an item that was hammered yesterday but is dead today keeps a huge count and never gets evicted ("cache pollution by stale popularity"). **Fix = aging:** periodically halve all counts, or use a *windowed/decaying* frequency (exponential decay). Redis's LFU uses an 8-bit logarithmic counter with a configurable decay (`lfu-log-factor`, `lfu-decay-time`).

6.3 FIFO / MRU

- **FIFO** evicts by insertion order regardless of use — a queue. Simple but can evict the hottest key just because it was inserted first. Rarely the right default.

- › **MRU** evicts the *most*-recently-used. Useful only for access patterns where "just touched it → won't touch again soon," e.g., scanning a file once front-to-back. Niche.

6.4 ARC (Adaptive Replacement Cache)

Used in ZFS and historically in PostgreSQL/IBM. Maintains **four lists**: T1 (recent, seen once), T2 (frequent, seen $\geq 2\times$), and *ghost* lists B1/B2 (recently-evicted keys, metadata only). Hits in the ghost lists tell ARC whether it's being hurt more by lack of recency or lack of frequency, and it **self-tunes the balance** between an LRU-ish and LFU-ish split. Scan-resistant because a one-time scan lands in T1 and is evicted before it can pollute T2.

6.5 W-TinyLFU --- what Caffeine uses

The modern state of the art (Caffeine, the default Java cache; also in CDN caches). Key ideas:

- › An **admission filter**: before a new item is admitted into the main cache, a tiny approximate-frequency sketch (a **Count-Min Sketch**) compares the *candidate's* estimated frequency to the *victim* it would evict. The candidate is admitted only if it's been requested more often. This **gates out one-hit-wonders** — scan resistance for nearly free.
- › An aging mechanism (periodic halving of the sketch counters) so old popularity decays.
- › A small **window LRU** in front to catch bursty recency.

Result: hit ratios close to Bélády-optimal across diverse workloads, with $O(1)$ ops and tiny metadata. **If asked "what does a production cache library use?" → W-TinyLFU (Caffeine).**

6.6 Scan resistance --- why it matters

A **scan** (e.g., a batch job reading every row once, or an analytics query) brings in thousands of keys that will each be used exactly once. Under plain **LRU**, these flood the cache and **evict the real hot set**, tanking your hit ratio right when load is highest. LFU, ARC, and W-TinyLFU resist this because a single access isn't enough to earn (or keep) a slot. This is a top reason production caches moved past naive LRU.

6.7 Redis maxmemory-policy options

When Redis hits maxmemory, the policy decides what to evict:

Policy	Behavior
noeviction	reject writes with an error (default for some setups) — protects data, risks errors
allkeys-lru	evict any key, approximated LRU
volatile-lru	evict LRU among keys with a TTL only
allkeys-lfu	evict any key, approximated LFU (Redis 4+)
volatile-lfu	LFU among keys with a TTL
allkeys-random	random key
volatile-random	random among TTL keys
volatile-ttl	evict the key with the shortest remaining TTL

Redis's LRU/LFU are **approximate** (sampled): it samples maxmemory-samples (default 5) random keys and evicts the best among them — $O(1)$ -ish, avoids maintaining a global linked list across millions of keys. With 10 samples it's near-exact.

Interview takeaway: Default to **LRU** and know the $O(1)$ hashmap+DLL implementation cold. Mention **LFU for scan resistance**, **W-TinyLFU/Caffeine** as the production answer, and that **Redis eviction is sampled/approximate**, not exact.

7 Cache Invalidation

"There are only two hard things in CS..." — this is the hard one. The job: keep the cache consistent enough with the source of truth, accepting a bounded staleness window you can defend.

7.1 1. TTL expiry --- the workhorse

Set an expiry; after it, the entry is gone and the next read repopulates. **Choose TTL by how stale you can tolerate**, not by gut feel.

- › Stock price: TTL ~1 s (or no cache).
- › Product details: TTL ~5 min.
- › User avatar URL: TTL ~1 hour.
- › Country list: TTL ~1 day.

Always add jitter (see avalanche). A fixed TTL set on a million keys at deploy time means a million simultaneous expiries later. $\text{Jitter} = \text{ttl} = \text{base} \pm \text{random}(0, \text{base} \times 0.1)$.

```
1 import random
2 def ttl_with_jitter(base_seconds, pct=0.1):
3     return int(base_seconds * (1 + random.uniform(-pct, pct)))
4 cache.set(key, val, ex=ttl_with_jitter(300)) # 270-330s
```

7.2 2. Write-time invalidation: DELETE, not UPDATE

On a DB write, **delete** the cache key so the next read repopulates from the fresh DB value. Do **not** try to update the cache in place with the new value.

Why delete beats update — the concurrent-writer race:

Two writers update user:42.

Time	Writer A (sets balance=100)	Writer B (sets balance=200)
t1	UPDATE db balance=100 (commit)	
t2		UPDATE db balance=200 (commit)
t3		SET cache user:42 = 200 ☒ correct so far
t4	SET cache user:42 = 100 ☒ (A was slow)	cache=100, DB=200 STALE

If both writers *update* the cache, their cache writes can land in the **opposite order** from their DB commits, leaving a stale value. With **delete**, the worst case is both delete the key (idempotent) and the next read repopulates the *current* DB value — self-correcting.

```
1 def update(key_id, changes):
2     db.update(key_id, changes) # source of truth first
3     cache.delete(f"user:{key_id}") # idempotent; next read refills fresh
```

Edge note: even delete has the *cache-aside stale-read race* (a slow reader refilling after the delete). TTL/leases/double-delete handle that residual window.

7.3 3. CDC / binlog-driven invalidation

Instead of the application remembering to invalidate, **tail the database's change log** (MySQL binlog, Postgres WAL via logical replication / Debezium) and emit invalidations.

```
DB write -> binlog -> Debezium/CDC -> Kafka topic -> invalidator service -> cache.delete(key)
```

- › **Pro:** decoupled — *any* writer (including ones that bypass the app, batch jobs, admin tools) triggers invalidation. Keeps **multiple services' caches** in sync from one source of truth. No “we forgot to invalidate” bugs.
- › **Con:** infrastructure + a small lag (ms–seconds). This is how large companies keep distributed caches coherent.

7.4 4. Versioned keys

Embed a version/etag in the key. Bump the version on change; old entries are simply never read again (and expire out via LRU/TTL).

```
key = f"product:{id}:v{version}"      # product:42:v7
# on change, increment version (stored e.g. in a tiny `version` row or another key)
# readers compute the current key from the current version -> old entries are orphaned
```

- › **Pro:** no delete needed; immutable entries are safe to cache forever and even to share across CDN edges. Atomic-feeling cutover.
- › **Con:** orphaned old versions linger until evicted (waste memory); need a place to store the current version.

7.5 5. Stale-while-revalidate (SWR)

Serve the **stale value immediately** while **asynchronously refreshing** in the background. The user never waits for a miss; the next user gets the fresh value.

```

def get_swr(key, ttl=300, stale_ttl=600):
    entry = cache.get_with_meta(key)          # {value, age}
    if entry is None:
        return load_and_fill(key, ttl)        # hard miss
    if entry.age <= ttl:
        return entry.value                    # fresh
    if entry.age <= stale_ttl:
        async_refresh(key, ttl)               # serve stale, refresh in bg
        return entry.value
    return load_and_fill(key, ttl)            # too stale -> block & refresh

```

Built into HTTP via Cache-Control: max-age=300, stale-while-revalidate=600. Excellent UX + natural stampede protection (only one background refresh).

7.6 The read-after-write inconsistency window

Under cache-aside, between “DB committed” and “cache deleted” (and the residual refill race), there's a window where a reader sees old data. Quantify it: usually **single-digit milliseconds** (delete latency) plus the rare refill race bounded by TTL. For “read-your-own-writes” (a user must see *their* edit immediately), route that user's next read to the DB (or write-through just their key) for a couple of seconds — don't rely on the cache to reflect their write instantly.

Interview takeaway: Delete, don't update, on writes (avoids the writer-ordering race). Use **TTL + jitter** as the universal backstop. For cross-service coherence, use **CDC/binlog**. For zero-wait freshness, **stale-while-revalidate**. Always state the **staleness window** explicitly.

8 The Thundering Herd & Friends (Failure Modes)

These four failure modes are interview gold. For each: the mechanism, the math, and the fix.

Cache Stampede	many concurrent misses for the SAME hot key → all hit the DB at once
Cache Penetration	queries for keys that DON'T EXIST → every one bypasses cache to DB
Cache Avalanche	MANY keys expire at the SAME TIME → DB load spike / collapse
Hot Key	one key gets disproportionate traffic → one shard/node melts

8.1 Cache stampede (thundering herd / dog-pile)

Mechanism: a single hot key (say a `product:viral` requested 50,000×/s) expires. For the brief moment the cache is empty, *all* 50,000 in-flight requests miss simultaneously, and **all 50,000 hit the DB** to recompute the same value. The DB, sized for ~5,000 QPS, falls over. The cache never gets repopulated fast enough, so it cascades.

Fix A — Single-flight / request coalescing (per-key lock). Only the *first* miss recomputes; everyone else waits and reads the freshly-filled value.

```

1 import threading
2 _locks = {} # key → Lock (use a striped lock map in prod)
3 _locks_guard = threading.Lock()
4
5 def _lock_for(key):
6     with _locks_guard:
7         return _locks.setdefault(key, threading.Lock())
8
9 def get_singleflight(key, ttl=300):
10     val = cache.get(key)
11     if val is not None:
12         return val
13     lock = _lock_for(key)
14     with lock: # only ONE thread per key proceeds
15         val = cache.get(key) # double-check: someone may have filled it
16         if val is not None:
17             return val
18         val = db.load(key) # the single DB hit
19         cache.set(key, val, ex=ttl_with_jitter(ttl))
20     return val

```

In a **distributed** setting the lock must be shared — use a Redis lock (`SET lock:key token NX PX 5000`). Other nodes that fail to grab the lock either wait-and-retry the cache, or serve a slightly stale value.

```

1 SET lock:product:viral <token> NX PX 5000 # acquire (NX = only if absent, PX = 5s expiry)
2 # ... load from DB, SET the real key ...
3 # release with a Lua compare-and-delete on <token> (see Redlock section)

```

Facebook's memcache solves this with **leases**: the first miss gets a lease token; concurrent missers are told "wait, someone's fetching." Only the lease holder may set the value.

Fix B — Probabilistic early expiration (XFetch). Refresh the value *before* it expires, with a probability that rises as expiry approaches, so exactly one request (statistically) refreshes early and the herd never forms.

```

1 import math, random, time
2 def xfetch(key, ttl, beta=1.0):
3     value, delta, expiry = cache.get_meta(key) # delta = time the last recompute took
4     now = time.time()
5     # recompute early if this draws past the expiry boundary
6     if value is None or now - delta * beta * math.log(random.random()) ≥ expiry:
7         start = time.time()
8         value = db.load(key)
9         delta = time.time() - start
10        cache.set_meta(key, value, delta, expiry=now + ttl)
11    return value

```

The $-\text{delta} * \text{beta} * \log(\text{random}())$ term makes expensive-to-recompute keys (large delta) refresh earlier and more eagerly. From the paper *"Optimal Probabilistic Cache Stampede Prevention."*

Fix C — Stale-while-revalidate (above): serve stale, one background refresh.

8.2 Cache penetration

Mechanism: requests for keys that **don't exist in the DB at all** (e.g., user:99999999, often malicious enumeration). Every such request misses the cache (correctly — nothing to cache) and falls through to the DB. The cache provides *zero* protection because there's nothing to cache. An attacker sending random non-existent IDs drives 100% of traffic to the DB.

Fix A — Negative caching. Cache the *absence* with a short TTL.

```

1 SENTINEL = "__MISS__"
2 def get_with_negcache(key, ttl=300, neg_ttl=30):
3     val = cache.get(key)
4     if val == SENTINEL:
5         return None # known-absent, served from cache
6     if val is not None:
7         return val
8     row = db.load(key)
9     if row is None:
10        cache.set(key, SENTINEL, ex=neg_ttl) # cache the miss (short TTL!)
11        return None
12    cache.set(key, serialize(row), ex=ttl)
13    return row

```

Keep the negative TTL **short** so a key that gets created later becomes visible quickly, and so an attacker can't fill memory with junk negative keys indefinitely.

Fix B — Bloom filter. A **Bloom filter** is a space-efficient probabilistic set: it answers "is this key *possibly* present?" with **no false negatives** (if it says "no," the key is definitely absent → reject without touching the DB) and tunable false positives (a "maybe" that's actually absent just costs one wasted DB lookup).

```

1 # Pre-populate a Bloom filter with all existing IDs (or maintain it on insert).
2 def get_with_bloom(key):
3     if not bloom.might_contain(key): # definitely absent → skip DB entirely
4         return None
5     return get_with_negcache(key) # maybe present → normal path

```

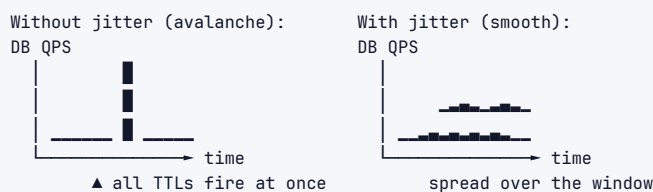

Bloom math: for n items and m bits with k hash functions, the false-positive rate is $p \approx (1 - e^{(-kn/m)})^k$, minimized at $k = (m/n) \ln 2$. Rule of thumb: **~10 bits per element** → **~1% false positive**; ~14.4 bits → ~0.1%. So 100M existing IDs at 1% FP needs ~120 MB — tiny vs protecting the DB. Redis has `BF.ADD` / `BF.EXISTS` (RedisBloom module). Downside: classic Bloom filters don't support deletes (use a **Counting** or **Cuckoo filter** if you must delete).

8.3 Cache avalanche

Mechanism: a *large set* of keys expire (or the cache restarts) at nearly the **same instant** → a flood of simultaneous misses overwhelms the DB. Common cause: bulk-loading the cache at deploy time with one fixed TTL → all keys expire together exactly TTL later. Or a Redis crash/restart → 0% hit ratio cold cache → DB takes 100% of read traffic and dies.

Fixes:

1. **TTL jitter** — randomize expiry ($\text{base} \pm 10\%$) so expiries are spread over time, not spiked. Single cheapest, most important fix.
2. **Staggered / gradual warmup** — on cold start, don't let all traffic hit the DB; ramp up, or pre-warm the top-N hot keys before taking traffic.
3. **Request coalescing** (above) so even simultaneous misses on the same key collapse to one DB hit.
4. **Circuit breaker / load shedding** in front of the DB so a cache failure degrades (serve errors/stale) rather than collapsing the DB.
5. **Multi-tier caching** — an L1 in-process cache survives an L2 (Redis) restart, smoothing the avalanche.



8.4 Hot key

Mechanism: a single key (a celebrity's profile, a viral product, a global config) gets a *disproportionate* share of traffic — say 30% of all reads on one key. In a sharded cache, that key lives on **one node**, so that one node's CPU/NIC saturates while others idle. The cache itself becomes the bottleneck (not the DB).

Fixes:

1. **Local (L1) cache** — cache the hot key in-process on every app server with a short TTL (1–5 s). 1,000 app servers → the hot key is read from the L2 only ~1,000×/s total instead of millions. Bounded staleness (the L1 TTL).
2. **Key replication / splitting** — store N copies under suffixed keys (`config:global#1 ... config:global#N`) and have each reader pick a random replica. Spreads load across N nodes. On write, update all N (or delete all N).
3. **Dedicated node / read replicas** for known hot keys.
4. **Client-side request coalescing** so each app server makes at most one upstream read per interval for the hot key.

```
1 def get_hot(key, fan=16):
2     replica = f"{key}#{random.randint(0, fan-1)}" # spread across fan nodes
```

```

3     val = cache.get(replica)
4     if val is None:
5         val = db.load(key)
6         cache.set(replica, val, ex=ttl_with_jitter(5))
7     return val

```

Interview takeaway: Memorize the four: **Stampede** (same key, mass miss → single-flight / SWR / probabilistic-early-expiry); **Penetration** (missing keys → negative cache + Bloom filter); **Avalanche** (mass simultaneous expiry → TTL jitter + warmup); **Hot key** (one key, one node → L1 + key splitting/replication). Naming the fix *before* being asked is the senior move.

9 Redis Deep Dive

Redis (REmote DIctionary Server) is the default distributed cache and far more — an in-memory data-structure server. Memcached is the simpler alternative.

9.1 Data structures (and what to cache with each)

Type	Description	Cache/use case
String	bytes up to 512 MB; ints (INCR)	serialized objects/JSON, counters, simple KV, rate-limit tokens
Hash	field→value map	a cached row/object by field (HSET user:42 name Alice age 30) — update one field without re-serializing
List	linked list, push/pop both ends	queues, recent-activity feeds, capped logs (LPUSH + LTRIM)
Set	unordered unique members	tags, unique visitors, set ops (intersection of followers)
Sorted Set (ZSet)	members ordered by score	leaderboards , priority queues, time-ranges (ZRANGEBYSCORE), rate limiting by timestamp
Stream	append-only log with consumer groups	event sourcing, durable pub/sub, work queues
HyperLogLog (HLL)	probabilistic cardinality, ~12 KB for billions	unique-visitor counts within ~0.81% error (PFADD/PFCOUNT)
Bitmap	bit ops on strings	daily active users (1 bit/user/day), feature flags, presence
Geo	geospatial (ZSet under the hood)	"drivers near me," radius queries

```

1 # Leaderboard with a sorted set:
2 ZADD game:leaderboard 5000 "alice" 4200 "bob" 6100 "carol"
3 ZREVRANGE game:leaderboard 0 9 WITHSCORES      # top 10
4 ZREVRANK game:leaderboard "bob"                  # bob's 0-based rank
5
6 # Cache a row as a hash (partial updates without re-serializing the whole object):
7 HSET user:42 name "Alice" age 30 city "NYC"
8 HGET user:42 name

```

```

9 EXPIRE user:42 300
10
11 # Atomic rate limiter (fixed window): N requests per 60s
12 INCR rl:user:42
13 EXPIRE rl:user:42 60 # set on first INCR; if INCR result > N → reject

```

9.2 Single-threaded model + pipelining

Redis executes commands on a **single thread** (the command-processing core; I/O threads exist in 6+ but command execution is serialized). Consequences:

- **No locks needed** for individual commands → each command is **atomic**. INCR, LPUSH, ZADD are race-free by construction. This is *why* Redis is great for counters and locks.
- **One slow command blocks everyone** — never run KEYS * or a giant SORT on prod; they stall the single thread. Use SCAN (cursor-based, incremental) instead.
- Throughput comes from doing each op in **microseconds**, so a single thread handles **100K–1M ops/sec**.

Pipelining amortizes network round trips by sending many commands without waiting for each reply:

```

1 Without pipelining: 1000 GETs = 1000 RTTs × 0.5 ms = 500 ms
2 With pipelining:    1000 GETs in one batch = ~1 RTT + processing ≈ 1–2 ms

```

MULTI/EXEC wraps commands in a transaction (queued, executed atomically, no interleaving). **Lua scripts** (EVAL) run server-side atomically — the right tool for read-modify-write logic (e.g., a correct rate limiter or a compare-and-delete lock release).

9.3 Persistence: RDB vs AOF

Redis is in-memory but can persist so a restart doesn't start cold (or lose write-back data).

	RDB (snapshot)	AOF (append-only file)
What	point-in-time binary dump (fork + copy-on-write)	logs every write command
Recovery	fast load (compact file)	slower replay, but more recent
Data loss on crash	up to the last snapshot interval (minutes)	tunable by fsync policy
Disk/CPU	periodic spike (fork)	continuous, file grows (needs rewrite/compaction)

AOF fsync policies (durability vs latency):

- `appendfsync always` — fsync every write: safest, slowest (each write waits for disk ~ms).
- `appendfsync everysec` — fsync once/second: **the common choice**; lose ≤1 s of writes on crash.
- `appendfsync no` — let the OS decide: fastest, can lose tens of seconds.

Many run **RDB + AOF together** (fast restart from RDB-style base + recent AOF tail). For a *pure cache*, persistence is often **off** — you accept a cold start and let the DB refill. Persistence matters when Redis holds write-back data or is used as a primary store.

9.4 Replication & Sentinel

- › **Replication:** a primary asynchronously streams writes to replicas. Replicas serve reads (scale reads) and act as failover targets. **Async** → **replication lag** (a read on a replica can be slightly stale).
- › **Sentinel:** monitors primary/replicas, performs automatic failover (promotes a replica) and tells clients the new primary. Provides HA for a non-clustered setup.

9.5 Redis Cluster --- 16384 hash slots

Horizontal scaling/sharding. The keyspace is partitioned into **16384 hash slots**; each key maps to a slot via $\text{CRC16}(\text{key}) \bmod 16384$. Each primary node owns a contiguous range of slots; clients are redirected (MOVED/ASK) to the node owning a key's slot.

```
slot(key) = CRC16(key) % 16384
Node A: slots 0-5460      Node B: slots 5461-10922      Node C: slots 10923-16383
```

- › **Resharding** moves slots (and their keys) between nodes; only the moved slots are affected (consistent-hashing-like minimal movement).
- › **Multi-key ops** (e.g., MGET, transactions) must touch keys in the *same* slot → use **hash tags**: {user:42}:profile and {user:42}:settings both hash on user:42, landing in the same slot.
- › 16384 ($= 2^{14}$) was chosen so the per-node slot bitmap in the cluster gossip messages stays small.

9.6 Distributed locks: Redlock and its criticism

A single-node lock: SET lock:resource <token> NX PX 10000 (acquire only if absent, auto-expire in 10 s). **Release must be a compare-and-delete** (only delete if the token is still yours) via Lua, else you might delete someone else's lock after your TTL expired:

```
1 -- release: delete only if the value still matches our token (atomic via Lua)
2 if redis.call("get", KEYS[1]) == ARGV[1] then
3     return redis.call("del", KEYS[1])
4 else
5     return 0
6 end
```

Redlock extends this across N independent Redis masters: acquire the lock on a majority ($\geq N/2 + 1$) within a time bound; the lock holds only if a majority granted it and enough TTL remains. **Martin Kleppmann's critique:** Redlock is unsafe for correctness-critical mutual exclusion because of clock drift, GC/STW pauses (a process can pause past its lease and act as if it still holds the lock), and timing assumptions. **Antirez (Redis author) rebutted** that with fencing it's fine for many uses. **Resolution:** for *efficiency* (avoid duplicate work) a Redis lock is fine; for *correctness* (must never have two holders), add a **monotonic fencing token** that the protected resource checks, or use a consensus system (ZooKeeper/etcd) built for this. Say this nuance and you've nailed the question.

9.7 Rate limiting in Redis

- › **Fixed window:** INCR key; EXPIRE key 60 — simple, but allows $2 \times$ burst at window edges.
- › **Sliding window log:** a ZSet of request timestamps; ZREMRANGEBYSCORE to drop old, ZCARD to count — accurate, more memory.
- › **Token bucket:** a Lua script that refills tokens based on elapsed time and decrements per request — smooth, the production standard.

9.8 Pub/Sub

SUBSCRIBE channel / PUBLISH channel msg — fire-and-forget messaging (no persistence; offline subscribers miss messages). Common for **cache invalidation broadcasts** ("delete key X everywhere"). For durable/replayable streams, use **Redis Streams** with consumer groups instead.

9.9 Redis vs Memcached

	Redis	Memcached
Data types	rich (string/hash/list/set/zset/stream/HLL/bitmap/geo)	strings/objects only
Threading	single-threaded core (atomic ops)	multi-threaded (scales on big multicore boxes)
Persistence	RDB + AOF	none (pure cache)
Replication/H	yes (Sentinel, Cluster)	none built-in (client-side sharding)
Eviction	many policies (LRU/LFU/TTL/random)	LRU (slab-based)
Use when	you need data structures, persistence, pub/sub, locks, HA	you want a <i>simple, blazing</i> , multi-threaded LRU string cache
Memory model	jemalloc; can fragment	slab allocator (fixed-size chunks; some waste, predictable)

Pick Memcached for a pure, huge, simple string cache on big boxes (Meta's classic use). **Pick Redis** for everything richer — which is most cases today.

Interview takeaway: Redis is single-threaded (→ atomic commands, but don't block it), shards via **16384 hash slots**, persists via **RDB+AOF** (everysec is the common fsync), and offers data structures that make leaderboards/rate-limits/locks trivial. Know the **Redlock critique** and the **fencing-token** answer.

10 CDN & HTTP Caching

The cache tiers *closest to the user* — often the highest-leverage of all because a hit here means **zero** origin/app/DB load and the lowest possible latency (served from a PoP physically near the user).

10.1 CDN (Content Delivery Network)

A globally-distributed network of **edge servers (PoPs)** that cache content near users (Cloudflare, Akamai, CloudFront, Fastly).

- › **What's cached:** static assets (images, JS, CSS, video segments) always; **cacheable dynamic** content (API responses with cache headers) increasingly.
- › **Cache key:** typically (host, path, query, selected headers). Tuning the key matters — including a ?utm=... tracking param in the key needlessly fragments the cache (cache busting); strip irrelevant query params.
- › **Origin shield:** a mid-tier cache between edges and origin so a global miss storm collapses to one origin fetch (stampede protection at CDN scale).
- › **TTL:** governed by Cache-Control: s-maxage / max-age from the origin, or CDN config.
- › **Purge / invalidation:** push-based — call the CDN API to purge a URL or a *cache tag* (e.g., purge everything tagged product-42). Purge-by-tag/surrogate-key is how you invalidate dynamic content at the edge. Purges propagate to all PoPs in seconds.

```
User (Tokyo) -> CDN PoP (Tokyo) —hit—> served in ~10 ms, origin untouched
└ miss -> Origin shield -> Origin (US) -> fill PoP
Next Tokyo user: hit.
```

10.2 HTTP caching headers --- the protocol

```
1 Cache-Control: public, max-age=3600, s-maxage=86400, stale-while-revalidate=60
2 ETag: "a1b2c3"
3 Vary: Accept-Encoding, Accept-Language
4 Age: 120
```

Cache-Control directives:

Directive	Meaning
max-age=N	fresh for N seconds (applies to browser and shared caches)
s-maxage=N	freshness for shared caches (CDN/proxy) only; overrides max-age there
no-cache	may store, but must revalidate with the origin before reuse (not "don't cache"!)
no-store	do not store at all (sensitive data — banking, PII)
private	only the browser may cache (per-user data); shared caches must not
public	any cache may store (even with auth headers)
immutable	won't change before expiry → skip revalidation (great for hashed asset URLs)
stale-while-revalidate=N	serve stale up to N s while revalidating in background

no-cache ≠ **no-store** — a classic gotcha. no-cache means "store it, but revalidate before each use." no-store means "never write it down." Use no-store for truly sensitive responses.

10.3 ETag + conditional requests → 304

An **ETag** is a content fingerprint (hash or version). The browser sends it back; if unchanged, the origin replies **304 Not Modified** with *no body* — saving bandwidth while still confirming freshness.

```
1 # First response:
2 HTTP/1.1 200 OK
3 ETag: "a1b2c3"
4 Cache-Control: no-cache           # must revalidate each time
5 <body...>
6
7 # Browser's revalidation request:
8 GET /resource HTTP/1.1
9 If-None-Match: "a1b2c3"
10
11 # Origin, if unchanged:
12 HTTP/1.1 304 Not Modified       # no body — browser reuses its cached copy
```

Last-Modified + If-Modified-Since is the timestamp-based alternative (weaker — 1-second granularity, and content can change without the mtime changing).

10.4 Vary and Age

- › **Vary** — tells caches which *request headers* differentiate the response. Vary: Accept-Encoding → store gzip and brotli variants separately. Vary: Accept-Language → per-language variants. Over-varying (e.g., Vary: User-Agent) **fragments the cache** and kills hit ratio.
- › **Age** — how long (seconds) the response has been sitting in caches. Age: 120 with max-age=3600 → 3480 s of freshness remain.

10.5 Browser vs CDN vs origin

```

Browser cache → private, per-user, max-age, ETag revalidation (zero network on hit)
↓ miss
CDN / edge   → shared, s-maxage, public assets, purge-by-tag, origin shield
↓ miss
Origin (app) → app L1 + Redis L2 + DB (everything earlier in this doc)

```

Interview takeaway: The cheapest cache hit is the one nearest the user. Master Cache-Control (max-age VS s-maxage, no-cache VS no-store, private VS public), ETag+If-None-Match → **304** for cheap revalidation, and Vary (don't over-vary). CDN invalidation = **purge by tag/surrogate-key**, not by guessing TTLs.

11 Consistency & Multi-Region

11.1 Staleness windows --- name them, bound them

Every cache introduces a window where readers may see old data. The senior skill is to *quantify and defend* it:

- › **TTL-bounded:** staleness $\leq$ TTL. A 5-min TTL = "reads may be up to 5 min stale." Pick TTL = max tolerable staleness.
- › **Invalidation-bounded:** staleness $\approx$ propagation delay of the delete/CDC event (ms-seconds), plus the refill race (TTL-bounded backstop).
- › **Replication-bounded:** with cache replicas, add replica lag.

State it as: *"Reads can be stale by up to ~5 min (TTL), or ~50 ms in the common case (delete propagation). For prices that's unacceptable, so we use a 2 s TTL there; for product descriptions 5 min is fine."*

11.2 Cache coherence across nodes/regions

A distributed cache (many Redis nodes, or per-app-server L1s) can hold **divergent copies** of the same key. After a write:

- › **L1 (in-process) caches** are the worst offenders — a delete must be **broadcast** to every app server (via Redis pub/sub or a message bus) or they serve stale until their local TTL. Keep L1 TTLs short (1–5 s) so divergence self-heals fast.
- › **Multi-region:** a write in region A must invalidate caches in regions B and C. Cross-region pub/sub adds latency (50–150 ms). Approaches:
 - **CDC/event stream** fanned out to all regions (durable, ordered).
 - **Versioned keys** — a global version bump means stale entries are simply never read (no race, no broadcast timing problem). Often the cleanest cross-region answer.
 - **Per-region TTL** as a backstop everywhere.

11.3 When NOT to cache

Caching is not free or always correct. **Don't cache** when:

- › **Strong consistency is required** and you can't tolerate any staleness (financial balances at point of trade, inventory at checkout for the last unit).
- › **Low reuse / high cardinality** — if each key is read ~once (uniform random access, unique per-request data), the hit ratio is near zero and you pay overhead for nothing (your hit-ratio math shows no benefit).
- › **Write-heavy with low read-back** — you'd thrash fills/invalidations.
- › **Highly personalized, non-shared data** that's cheap to compute.
- › **Security-sensitive responses** at shared caches (set private/no-store).
- › **Tiny/fast sources** — caching adds a hop and a consistency problem for no latency win.

Interview takeaway: Always say the **staleness window** out loud and tie the TTL to business tolerance. For multi-region, **versioned keys / CDC** beat TTL guesses. And know **when not to cache** — naming that is a maturity signal.

12 Designing a Cache (Worked Example)

Prompt: "Design caching for a product catalog + a hot leaderboard for an e-commerce flash-sale site. Reads ~200K QPS, writes ~2K QPS. DB (Postgres) can serve ~10K QPS comfortably."

12.1 1. Clarify access patterns & SLOs

- › **Reads** $\square$ **writes** (100:1) → caching is the obvious lever.
- › Catalog reads are **shareable** (every user sees the same product) → high reuse, ideal.
- › **Skew:** a flash sale means a handful of products are *viral* (hot keys) and bursts cause stampede/avalanche risk at sale start.
- › SLO: p99 product read < 20 ms; DB must stay < 10K QPS.

12.2 2. Choose tiers

```

Browser/CDN (static assets, images)      - s-maxage=86400, immutable hashed URLs
  ↓ miss
App-local L1 (Caffeine, W-TinyLFU)       - top hot products, TTL 5s, 50k entries
  ↓ miss
Redis L2 (cluster, cache-aside)          - all products, TTL 5min + jitter
  ↓ miss
Postgres (source of truth, read replicas)

```

12.3 3. Keys, values, TTLs

Data	Key	Value	TTL	Pattern
Product detail	product:{id}:v{ver}	JSON (hash)	5 min $\pm$ 10% jitter	cache-aside
Price (volatile)	price:{id}	string	5 s	cache-aside + CDC invalidation
Leaderboard	sale:leaderboard	ZSet (ZADD/ZREVRANGE)	live, no TTL	write-through to ZSet
Inventory (last units)	—	not cached at checkout	—	DB with row lock

12.4 4. Invalidation

- Product edits → **delete product:{id}** (or bump version → :v{ver}). Use **CDC (Debezium → Kafka)** so admin tools / batch price updates also invalidate, across regions.
- Price changes are frequent → short TTL (5 s) **plus** CDC delete for immediacy. Read-your-own-writes for sellers: route their next read to the DB for 2 s.
- Inventory at the *last unit* → **don't cache**; correctness > latency (avoid overselling).

12.5 5. Stampede / avalanche / hot-key protection

- Sale start avalanche:** pre-warm the top-200 product keys **before** opening the sale; jitter TTLs so they don't expire together.
- Viral product stampede: single-flight** (Redis SET NX lock) on miss for hot keys; or **stale-while-revalidate** so the herd is served stale during a refresh.
- Hot key (the one viral product at 30% of traffic): L1 cache** on every app server (5 s TTL) → the millions of reads collapse to ~(#app-servers) reads/s on Redis; optionally **key-split** the Redis entry across replicas.
- Penetration** (bots hitting random/expired product IDs): **negative cache** (30 s) + a **Bloom filter** of valid product IDs (~10 bits/ID).

12.6 6. Expected hit ratio & load math

With CDN+L1+L2 and a Zipfian catalog, assume **L2 hit ratio ≈ 96%**.

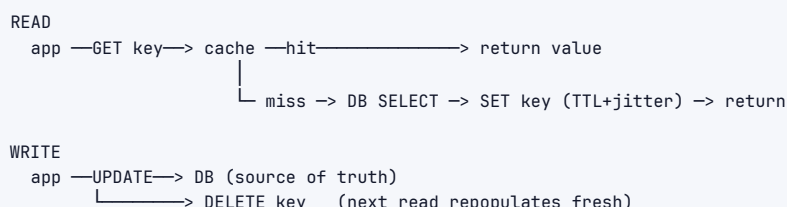
```
1 DB read QPS = (1 - 0.96) × 200,000 = 8,000 QPS < 10,000 ✓ (within budget)
2 Effective read latency ≈ 0.96 × 0.5ms + 0.04 × 20ms = 0.48 + 0.8 = 1.28 ms ✓
```

Push L2 to **98%** (better warmup/jitter/longer TTL where safe) → DB sees **4,000 QPS**, halving DB load and giving headroom for traffic spikes. The L1 tier further shields Redis from the hot keys. The leaderboard is a single Redis ZSet — ZREVRANGE 0 9 is microseconds and never touches the DB.

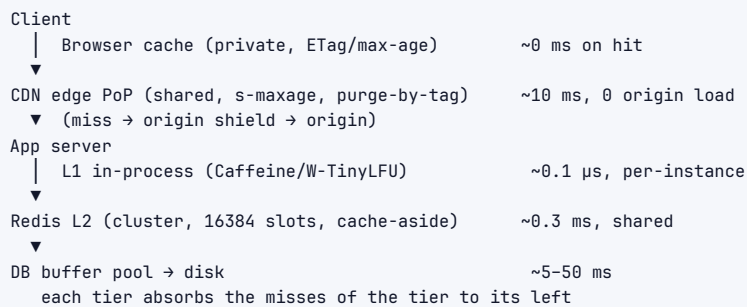
Interview takeaway: Drive the design from **access pattern** → **tiers** → **keys/TTL** → **invalidation** → **failure-mode protection** → **hit-ratio/load math**. Ending with "this keeps the DB at 8K QPS under its 10K budget" turns a hand-wave into an engineering answer.

13 Architecture / Diagrams

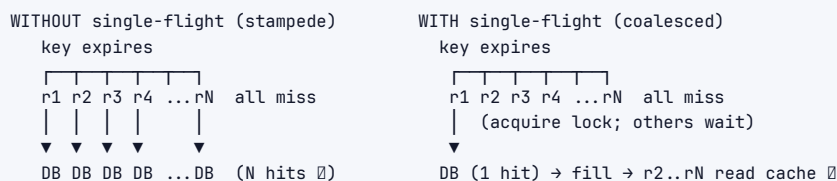
13.1 Cache-aside read/write flow



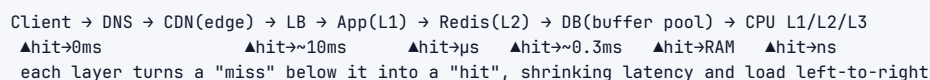
13.2 Multi-tier cache hierarchy



13.3 Stampede: herd vs single-flight



13.4 Where caches live (request path)



14 Real-World Examples

- Meta / memcached in front of MySQL** — the *"Scaling Memcache at Facebook"* paper: a giant look-aside cache layer. Introduces **leases** (a token issued on miss) to (a) prevent stampedes — only the lease holder fetches — and (b) prevent stale sets — a write invalidates outstanding leases so a slow refill is rejected. Regional pools and a "mcrouter" proxy route and replicate.
- Netflix EVCache** — a Memcached-based, **multi-region replicated** cache. Each region holds a full copy of hot data so a regional failover still serves warm; writes replicate cross-region asynchronously. Sized for resilience, not just hit ratio.
- Amazon DynamoDB Accelerator (DAX)** — a write-through/read-through cache *in front of* *DynamoDB* giving microsecond reads; ElastiCache (Redis/Memcached) for general caching; CloudFront for the edge.
- Twitter/X timelines** — Redis (sorted sets) cache precomputed timelines (fan-out-on-write). Celebrity posts use **fan-out-on-read** (hot-key avoidance) rather than invalidating 100M follower caches.
- CDNs (Cloudflare/Akamai/Fastly)** — the entire product is edge caching: static assets, plus cacheable dynamic content with **purge-by-surrogate-key**. Fastly's instant purge is famous (sub-150 ms global).
- CPU caches (L1/L2/L3)** — the same locality principle in hardware: a 64-byte cache line, LRU-ish replacement, and the reason cache-friendly memory access patterns can beat a better Big-O with poor locality.
- DNS resolvers** — cache A/AAAA records with TTLs; a TTL-driven cache that the whole internet depends on.

15 Real-Life Analogies

One household kitchen — every concept is a shelf, a habit, or a chore.

Concept	Analogy
Cache	Your kitchen counter — the few ingredients you use daily kept within arm's reach; the pantry/supermarket (DB) holds everything but is slower to reach
Hit / miss	Reaching for salt: it's on the counter (hit) vs you have to walk to the pantry (miss)
Hit ratio	The fraction of times what you need is already on the counter — the better stocked for <i>your</i> habits, the fewer pantry trips
TTL	The milk's expiry date — past it you stop trusting the carton and go back to the store (source of truth)
Cache-aside	" Check the fridge first ; if it's empty, go shopping and <i>then</i> restock the fridge" — and you only stock what you actually went looking for
Write-through	Every time you buy milk you immediately put it both in the fridge and note it on the shopping list — always consistent, but a little extra work each time
Write-back	Jotting purchases on a sticky note and updating the master list later — fast now, but if you lose the sticky note before copying it, those purchases are gone
Write-around	Buying a bulk sack of flour and putting it straight in the pantry , skipping the counter — it won't be used today, no point cluttering the counter
LRU	When the counter is full, clear off whatever you haven't touched in the longest to make room
LFU	Keep the things you use most often (the everyday salt and oil) even if you didn't touch them in the last hour; toss the gadget you used once
Scan pollution	Hosting a one-off dinner party floods the counter with specialty items you'll never use again, shoving your daily staples into the pantry — LFU/W-TinyLFU refuse to let a one-time guest evict your staples
Invalidation (delete on write)	When a recipe changes, you throw out the old prepped portion rather than trying to edit it in place — next time you re-prepare from the current recipe
TTL jitter	Staggering expiry dates so not every carton spoils on the same Monday , sparing you one giant emergency shopping trip
Cache stampede	The milk expires at breakfast and the whole family bolts to the store at once ; a single "I'll go, everyone wait" (single-flight) sends one person instead of five
Cache penetration	A kid keeps asking for an ingredient you've never owned ; you put a sticky note "we don't have unicorn jam" (negative cache) so you stop checking the empty pantry each time
Cache avalanche	The whole fridge expires the same day , forcing a full restock all at once — staggering dates (jitter) prevents the pile-up
Hot key	Everyone wants the one popular sauce ; you keep a small bottle on <i>every</i> counter (L1 replication) so they don't all crowd the single pantry shelf
CDN	Corner convenience stores stocking popular items near each neighborhood, so people don't all drive to the central supermarket
ETag / 304	You phone the store: "is the recipe card still version a1b2c3?" — "yep, unchanged" — so you don't re-copy the whole card , just trust your copy
Stale-while-revalidate	You use yesterday's bread right now while a fresh loaf bakes in the background — no one waits hungry
Bloom filter	A doormat sign listing what's definitely not in the house — if it's on the "we never stock this" sign, don't even open the pantry

16 Memory Tricks / Mnemonics

- › **Patterns — "Aside, Through, Back, Around":** lazy / sync / async / bypass.
- › **Cache-aside rule — "Read fills, Write deletes."** (Never *update* the cache on write.)
- › **The four failure modes — "SPAH"** (Stampede, Penetration, Avalanche, Hot-key) and their fixes:
 - Stampede → **S**ingle-flight (+ SWR / probabilistic early expiry)
 - Penetration → **P**robabilistic filter (Bloom) + negative cache
 - Avalanche → **A**dd jitter (+ warmup)
 - Hot-key → **H**ave a local L1 (+ key splitting)
- › **"Two hard things: cache invalidation and naming things."**
- › **Eviction — "LRU = recency, LFU = frequency, W-TinyLFU = the smart one (Caffeine)."**
- › **Hit-ratio intuition — "90→99% matters more than 0→90%"** (the tail dominates effective latency).
- › **HTTP — "no-cache stores-but-asks; no-store never writes."**
- › **Redis sharding — "16384 slots, CRC16 mod."**
- › **Durability — "RDB = snapshot, AOF = journal; everysec is the sweet spot."**

17 Common Interview Questions

17.1 Q1: Explain cache-aside, and when would you choose it over write-through?

Model answer: In **cache-aside**, the app checks the cache; on a miss it reads the DB and *fills* the cache, and on writes it updates the DB and **deletes** the key (not update — deletion avoids the writer-ordering race). It's the default because it's simple and resilient: if the cache is down, reads fall through to the DB, so a cache outage degrades latency, not correctness. Use **write-through** (write cache+DB synchronously) when reads must be fresh immediately after a write and you can tolerate higher write latency — but it wastes memory caching data that may never be read and only caches keys that have been written. Cache-aside suits read-heavy workloads with tolerable staleness; write-through suits low-staleness-tolerance, read-heavy data like config.

Follow-ups:

- › *"Why delete instead of update the cache on write?"* → Two concurrent writers can apply their cache writes in the opposite order to their DB commits, leaving a stale value. Delete is idempotent and self-correcting — the next read refills the current value.
- › *"What's the residual race even with delete?"* → A slow reader that read the old DB value before the writer's commit can fill the cache with stale data *after* the delete. Bound it with TTL, or close it with leases / double-delete.

17.2 Q2: How do you keep a cache consistent with the database?

Model answer: You can't make it perfectly consistent cheaply, so you bound the staleness. Options, in increasing strength/cost: **TTL** (accept staleness $\leq$ TTL — always add jitter); **delete-on-write** (next read refills); **CDC/binlog invalidation** (tail the DB change log → emit deletes, so even non-app writers and other services stay coherent); **versioned keys** (bump a version; stale entries are orphaned, never read — great cross-region); **stale-while-revalidate** for zero-wait reads. I'd pick TTL+delete for a single service, and CDC or versioned keys for cross-service/cross-region coherence — and I'd state the staleness window explicitly (e.g., " ≤ 5 min via TTL, ~50 ms typical via delete propagation").

Follow-ups:

- › *"User must see their own edit immediately?"* → Read-your-own-writes: route that user's reads to the DB (or write-through their key) for a couple of seconds after their write.

17.3 Q3: What is a cache stampede (thundering herd) and how do you prevent it?

Model answer: A **stampede** happens when a single hot key expires and many concurrent requests all miss and hit the DB at once to recompute the same value — at, say, 50K QPS on one key, that's 50K simultaneous DB queries that can topple the DB. Prevent it with **single-flight / request coalescing**: a per-key lock (in distributed setups, a Redis SET NX lock) so only the first miss recomputes while others wait and then read the filled value. Alternatives: **stale-while-revalidate** (serve the old value, one background refresh) and **probabilistic early expiration** (refresh slightly before TTL with rising probability, so the herd never forms). For *mass* simultaneous expiry, that's avalanche → add **TTL jitter**.

Follow-ups:

- › *"How does Facebook do it?"* → Leases: on a miss the cache issues a lease token; only the holder may set the value, and a concurrent write invalidates the lease, also preventing stale sets.

17.4 Q4: How do you protect against queries for keys that don't exist (penetration)?

Model answer: Those queries (often malicious ID enumeration) bypass the cache because there's genuinely nothing to cache, so 100% hit the DB. Two defenses: **negative caching** — cache a sentinel "absent" value with a *short* TTL (so a later-created key appears quickly and attackers can't bloat memory); and a **Bloom filter** of valid keys in front — it has no false negatives, so a "definitely not present" answer rejects the request without a DB hit. Bloom math: ~10 bits/element → ~1% false positives, so 100M IDs ≈ 120 MB to shield the DB. Combine both for robustness.

Follow-ups:

- › *"Bloom filter can't delete — what if a key is removed?"* → Use a Counting Bloom or Cuckoo filter, or rebuild periodically; a false-positive just costs one wasted DB lookup, which is safe.

17.5 Q5: LRU vs LFU --- implement LRU and explain when each wins.

Model answer: **LRU** evicts the least-recently-used and captures temporal locality; implement O(1) with a hash map (lookup) + doubly-linked list (recency): on access move the node to the head, evict from the tail. **LFU** evicts the least-frequently-used; it keeps genuinely popular items and resists **scan pollution** (a one-time bulk read won't evict the long-term hot set, as it would under LRU), but it needs frequency counters and **aging** (decay counts) so yesterday's hot item doesn't stick forever. Production caches use **W-TinyLFU** (Caffeine) — a frequency-gated admission filter over a windowed LRU — for near-optimal hit ratios with scan resistance.

Follow-ups:

- › *"How does Redis do LRU?"* → Approximate/sampled: it samples maxmemory-samples random keys and evicts the best, avoiding a global linked list across millions of keys.

17.6 Q6: Where would you place caches in a web architecture, and what does each buy you?

Model answer: Layered, each absorbing the misses of the one in front: **browser/HTTP cache** (free, nearest the user, via ETag/Cache-Control, zero network on a hit); **CDN** at the edge for static and cacheable dynamic content (zero origin load on a hit); **app-local L1** (in-process, microseconds, per-instance — great for hot keys and shielding Redis); a **distributed L2 (Redis)** shared across instances for hot rows/sessions/leaderboards; and the **DB buffer pool**. Each tier shrinks latency and downstream load left-to-right; an L1 also survives an L2 restart, smoothing avalanches.

17.7 Q7: Walk me through Redis internals you'd rely on for a cache.

Model answer: Redis is an in-memory data-structure server with a **single-threaded** command core, so each command (INCR, ZADD, SET NX) is **atomic** — ideal for counters, rate limiters, and locks — but you must never run blocking commands like KEYS * (use SCAN). **Pipelining** amortizes RTTs (1000 GETs in ~1 RTT). It shards via **16384 hash slots** ($\text{CRC16}(\text{key}) \bmod 16384$) in Cluster mode; co-locate related keys with **hash tags** {user:42}. Persistence is **RDB** (snapshots) and/or **AOF** (write log; appendfsync everysec is the usual durability/latency sweet spot) — though a pure cache often runs without persistence and accepts a cold start. Rich types (hashes for rows, sorted sets for leaderboards, HLL for cardinality, bitmaps for DAU) let me cache the right shape directly.

Follow-ups:

- › “Redis vs Memcached?” → Memcached is multi-threaded, strings-only, no persistence — a simple blazing LRU for big boxes; Redis adds data structures, persistence, replication/cluster, pub/sub, and locks. Pick Redis unless you specifically want Memcached's simplicity/multi-threading.

17.8 Q8: How do distributed locks work in Redis, and what are the caveats?

Model answer: A single-node lock is SET lock:res <token> NX PX 10000 (acquire only if absent, auto-expire), released by a **Lua compare-and-delete** so you only release *your* token (never delete a lock that already expired and was re-acquired). **Redlock** extends this to a majority of N independent masters for HA. The caveat (Kleppmann): it's unsafe for *correctness*-critical mutual exclusion because of clock drift and GC/STW pauses — a process can pause past its lease and still think it holds the lock. So for *efficiency* (dedupe work) a Redis lock is fine; for *correctness* add a monotonic **fencing token** that the protected resource validates, or use ZooKeeper/etcd. State that nuance.

17.9 Q9: Explain HTTP caching: Cache-Control, ETag, and what 304 saves.

Model answer: Cache-Control governs caching: max-age (browser+shared), s-maxage (shared/CDN only), private (browser only — per-user data), public, no-cache (store but **revalidate** before use), no-store (never store — sensitive data), immutable (skip revalidation). An **ETag** is a content fingerprint; the browser revalidates with If-None-Match, and if unchanged the origin returns **304 Not Modified** with **no body** — confirming freshness while saving the bandwidth of resending the payload. Vary declares which request headers split cache variants (don't over-vary — it fragments the cache). s-maxage lets a CDN cache aggressively while the browser revalidates more often.

Follow-ups:

- › “no-cache vs no-store?” → no-cache = store but revalidate each use; no-store = never write it down (for truly sensitive responses).

17.10 Q10: Design caching for a product catalog under a flash sale.

Model answer: (see the Worked Example) Tiers: CDN for images, L1 (Caffeine) for hot products with a 5 s TTL, Redis L2 cache-aside for all products with a 5 min TTL + jitter, Postgres + read replicas as source of truth. Keys product:{id}:v{ver}; prices on a short 5 s TTL + CDC invalidation. Pre-warm top products before the sale (avalanche), single-flight or SWR on viral keys (stampede), L1 replication for the one hot product (hot key), Bloom + negative cache for bot-driven non-existent IDs (penetration), and **don't cache the last-unit inventory** (correctness > latency). At ~96% L2 hit ratio, the DB sees $(1 - 0.96) \times 200K = 8K$ QPS, under its 10K budget — push to 98% for headroom.

17.11 Q11: Your cache just went down in production. What happens, and how should the system behave?

Model answer: Ideally the system **degrades, not fails**: cache-aside means reads fall through to the DB, so correctness is preserved but the DB now takes 100% of read traffic — which can overwhelm it (a cold-cache avalanche). Protections: a **circuit breaker / load shedder** in front of the DB, an **L1 in-process cache** that survives the L2 outage, **request coalescing** so simultaneous misses collapse, and **gradual warmup** rather than full traffic on a cold cache. The anti-pattern is depending on the cache for correctness (e.g., write-back losing un-flushed writes) — that turns a performance incident into a data-loss incident.

Follow-ups:

- › “How do you warm it back up?” → Pre-load the top-N hot keys from the DB before taking full traffic, or shadow/ramp traffic; jitter TTLs so the refilled keys don't avalanche later.

17.12 Q12: How do you choose a TTL?

Model answer: TTL = the **maximum staleness the business tolerates** for that data, not a gut number. Stock price ~1 s (or don't cache), inventory near-zero, not cached; product detail ~5 min; avatar ~1 hour; country list ~1 day. **Always add jitter** ($\pm 10\%$) to avoid avalanche, and combine TTL with **event-driven invalidation** for freshness-critical keys so you get immediacy *and* a TTL backstop. Shorter TTL = fresher but lower hit ratio and more DB load; longer TTL = higher hit ratio but staler — quantify both with the hit-ratio/load formulas.

18 Senior-Level Discussion Points

- › **Caching is a correctness decision, not just performance.** Every cache adds a staleness window you must defend *per use case*. A stale price can cause a mispriced trade; a stale avatar is harmless. Senior engineers tie TTL/invalidation to business tolerance, not convenience.
- › **Hit ratio is the lever — and it's an Amdahl's-Law curve.** The jump from 90→99% matters far more than 0→90% because misses dominate effective latency and DB load as $h \rightarrow 1$. Model the access distribution (Zipf) before claiming a cache helps; uniform-random/low-reuse workloads barely benefit.
- › **The cache as source of truth is a trap.** Write-back can lose un-flushed data; depending on the cache for durability converts a perf incident into data loss. DB stays authoritative; cache is allowed to forget.
- › **Stampede protection must be designed in, not bolted on.** Single-flight, SWR, probabilistic early expiration, and Facebook leases each have trade-offs (latency vs staleness vs complexity). At CDN scale, an **origin shield** is the same idea.
- › **Multi-region invalidation is genuinely hard.** TTL guesses are fragile across regions; CD-C/event streams and **versioned keys** (no timing race) are the robust answers. Quantify cross-region propagation (50–150 ms).
- › **L1 vs L2 trade-off.** An in-process L1 gives microsecond hits and shields the L2/DB from hot keys and L2 outages, but multiplies the **coherence** problem (every instance has its own copy) — keep L1 TTLs short and broadcast invalidations.
- › **Observability is the early-warning system.** Track **hit ratio, eviction rate, p99, key cardinality, memory fragmentation**. A *falling hit ratio* often precedes an incident (working set outgrew memory, or a scan is polluting LRU → consider W-TinyLFU).
- › **Memory accounting & fragmentation.** Redis memory $\neq$ data size (jemalloc fragmentation, per-key overhead ~50–90 bytes). Watch `mem_fragmentation_ratio`; huge numbers of tiny keys waste memory — consider hashing/packing.
- › **Serialization cost is real.** A “fast” Redis cache can be dominated by JSON (de)serialization

on the app side. Use compact formats (MessagePack/protobuf) and cache the *deserialized* shape in L1.

19 Typical Mistakes Candidates Make

1. **"Just add Redis"** — no pattern, TTL, invalidation, or stampede story. The details are the whole answer.
2. **Updating the cache on write** instead of **deleting** the key — introduces the writer-ordering stale race.
3. **Uniform TTLs** → avalanche. Forgetting **jitter** is the single most common omission.
4. **No stampede protection** for hot keys — assuming a TTL expiry is harmless when it's a thundering herd.
5. **Ignoring penetration** — not handling non-existent-key floods (negative cache + Bloom filter).
6. **Caching everything** — low reuse → near-zero hit ratio, wasted memory; cache the *hot set*, prove it with the hit-ratio math.
7. **Treating the cache as the source of truth** — write-back data loss; depending on it for correctness.
8. **Ignoring cache-down behavior** — a good design degrades (falls through to DB with a circuit breaker), it doesn't fail or melt the DB cold.
9. **Confusing no-cache with no-store** in HTTP — one revalidates, the other never stores.
10. **Forgetting the cold-start avalanche** after a deploy/restart (0% hit ratio → DB takes everything) — pre-warm.
11. **Over-Varying** HTTP responses (Vary: User-Agent) — fragments the CDN cache, destroying hit ratio.
12. **Claiming Redlock gives correctness** — without mentioning clock drift / GC-pause caveats and fencing tokens.

20 How This Connects to Other Topics

Topic	Connection
System Design	The #1 read-scaling tool; pairs with read replicas, CDNs, fan-out-on-write. "DB is the bottleneck → add a cache" — then the details decide your score.
Databases (§08)	Buffer pool <i>is</i> a cache; query/result caching; CDC/binlog (WAL/logical replication) drives invalidation; cache-aside complements read replicas.
Performance Engineering (§09)	The latency ladder is <i>why</i> caching wins; hit-ratio math is Amdahl-style; cache locality (L1/L2/L3) is the hardware analog; stampede = thread-pool/DB saturation.
Networks (§03)	HTTP caching (ETag, Cache-Control, 304), CDNs at the edge, DNS TTL caching.
Distributed Systems	Cache coherence, multi-region invalidation, distributed locks (Redlock + fencing), consistency models (eventual, read-your-own-writes).
Concurrency	Single-flight/request coalescing = a per-key lock; Redis's single-threaded atomicity; false sharing in CPU caches.
Data Structures	LRU (hashmap + DLL), LFU (freq lists), Bloom/Cuckoo filters, Count-Min Sketch (W-TinyLFU), skip lists (Redis ZSet), HyperLogLog.
API Design (§14)	Cache-Control/ETag, idempotency keys, conditional requests.

21 FAANG Interview Tips

1. **When you say "add a cache," immediately specify pattern + TTL + invalidation.** "Cache-aside on product:{id}, 5-min TTL with 10% jitter, delete-on-write, single-flight on miss" — that one sentence is the senior signal.
2. **Name a failure mode and its fix before being asked** — stampede→single-flight/SWR, avalanche→jitter, penetration→Bloom/negative-cache, hot-key→L1/key-split.
3. **Always estimate the hit ratio** from the access pattern and convert it to **DB load reduction** ("96% → DB sees 8K of 200K QPS"). Numbers turn hand-waving into engineering.
4. **State the staleness window explicitly** and tie the TTL to business tolerance.
5. **Be explicit that the DB stays the source of truth** and the cache **degrades gracefully** (fall-through + circuit breaker), never causing data loss.
6. **Know the LRU O(1) implementation cold** (hashmap + DLL) and name **W-TinyLFU/Caffeine** as the production-grade answer.
7. **For Redis**, mention single-threaded atomicity, 16384 hash slots, RDB/AOF, and the **Redlock caveat + fencing token**.
8. **For the edge**, distinguish max-age vs s-maxage, no-cache vs no-store, and ETag→304; mention purge-by-tag for CDN invalidation.
9. **Know when *not* to cache** (strong consistency, low reuse, security-sensitive) — naming it shows maturity.
10. **Use the latency ladder** to justify the cache quantitatively (RAM ~100 ns vs DB ~20 ms = ~200,000×).

22 Revision Cheat Sheet

22.1 10-Minute Summary

A **cache** trades freshness for speed, working because of **locality** (a hot minority of keys serves most traffic). Effectiveness = **hit ratio**: $T_{\text{eff}} = h \cdot t_{\text{cache}} + (1-h) \cdot t_{\text{db}}$, and DB load = $(1-h) \cdot \text{QPS}$. The 90→99% range matters most.

Patterns: **cache-aside** (default — read fills, write *deletes*; resilient), **write-through** (sync, always fresh, slower writes), **write-back** (async, fast, data-loss risk), **write-around** (skip cache on write), **read-through** (cache lib loads on miss).

Eviction: **LRU** (recency; O(1) hashmap+DLL), **LFU** (frequency; scan-resistant, needs aging), FIFO/MRU/Random, **ARC** and **W-TinyLFU** (Caffeine — production-grade, scan-resistant). Redis eviction is **sampled/approximate**.

Invalidation: **TTL + jitter** (backstop), **delete-on-write** (not update — avoids writer-ordering race), **CDC/binlog** (cross-service coherence), **versioned keys** (cross-region, no timing race), **stale-while-revalidate** (zero-wait). Mind the cache-aside read-after-write race; bound it with TTL/leases/double-delete.

Failure modes (SPAH): **Stampede** → single-flight / SWR / probabilistic early expiry / leases; **Penetration** → negative cache + Bloom filter (~10 bits/elem → 1% FP); **Avalanche** → TTL jitter + warmup; **Hot key** → L1 + key splitting/replication.

Redis: single-threaded atomic commands, pipelining, RDB+AOF (everysec), replication/Sentinel, Cluster with **16384 hash slots** (CRC16 mod), rich types (ZSet leaderboards, HLL, bitmaps), Redlock + **fencing tokens**. Memcached = simpler, multi-threaded, strings-only.

Edge/HTTP: CDN edge caching + purge-by-tag; Cache-Control (max-age/s-maxage/no-cache≠no-store/private) ETag+If-None-Match→**304**, Vary (don't over-vary), Age.

22.2 Key Numbers to Know

- › RAM ~100 ns vs Redis/LAN ~0.3–0.5 ms vs DB ~5–50 ms (cache is 100–1000× faster than DB).
- › Hit ratio → effective latency: 90% ≈ 8× speedup; 99% ≈ 29×.
- › DB load = (1-h) · QPS: at h=0.96, 200K read QPS → 8K DB QPS.
- › Bloom filter: ~10 bits/element → ~1% false positive; 100M IDs ≈ 120 MB.
- › Redis: 16384 hash slots; ~100K–1M ops/sec single thread; everysec AOF loses ≤1 s.
- › Cache line: 64 bytes (CPU caches).

22.3 Cheat-Sheet Table

Concept	One-liner
Cache-aside	read fills, write deletes key (default, resilient)
Write-through	write cache+DB sync; always fresh, slower writes
Write-back	write cache, async→DB; fast, risks loss
Write-around	write DB, skip cache; avoids churn
Read-through	cache library loads on miss
LRU / LFU	recency / frequency; LFU is scan-resistant
W-TinyLFU	Caffeine; admission filter + windowed LRU (production)
TTL + jitter	expiry with ±10% randomization (dodge avalanche)
Delete > Update	avoids the concurrent-writer stale race
CDC / binlog	event-driven invalidation; cross-service coherence
Versioned keys	:v7; stale entries orphaned (cross-region)
Stale-while-revalidate	serve stale, refresh in background
Stampede	hot-key mass miss → single-flight / SWR / leases
Penetration	missing-key flood → negative cache + Bloom filter
Avalanche	mass expiry → TTL jitter + warmup
Hot key	one key/one node → L1 + key splitting
Hit ratio	hits/(hits+misses); the lever that matters
Redis sharding	16384 slots, CRC16(key) mod 16384
Redis persistence	RDB snapshot + AOF journal; everysec
Redlock	OK for efficiency; for correctness add fencing token
HTTP	no-cache=revalidate, no-store=never; ETag→304
Tiers	browser → CDN → L1 → Redis L2 → DB buffer pool → CPU L1/2/3

Golden rule: Cache the **hot set** with a clear **pattern + TTL (jittered)**, **invalidate by deleting keys** (and CDC/versioning for coherence), **protect against stampede/penetration/avalanche/hot-key**, keep the **DB as source of truth**, and always state the **hit-ratio** and **staleness window** out loud.

Last updated: 2026-06-17 / Topic: Caching / Level: FAANG Senior